EN3150 Pattern Recognition
Additive Models

Most of the contents are based on

1. C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

Presented by Sampath K. Perera

Decision Tree

Predictive performance: how well a machine learning model can accurately predict outcomes based on input data (Accuracy, Precision, Recall (Sensitivity), F1 Score,…)

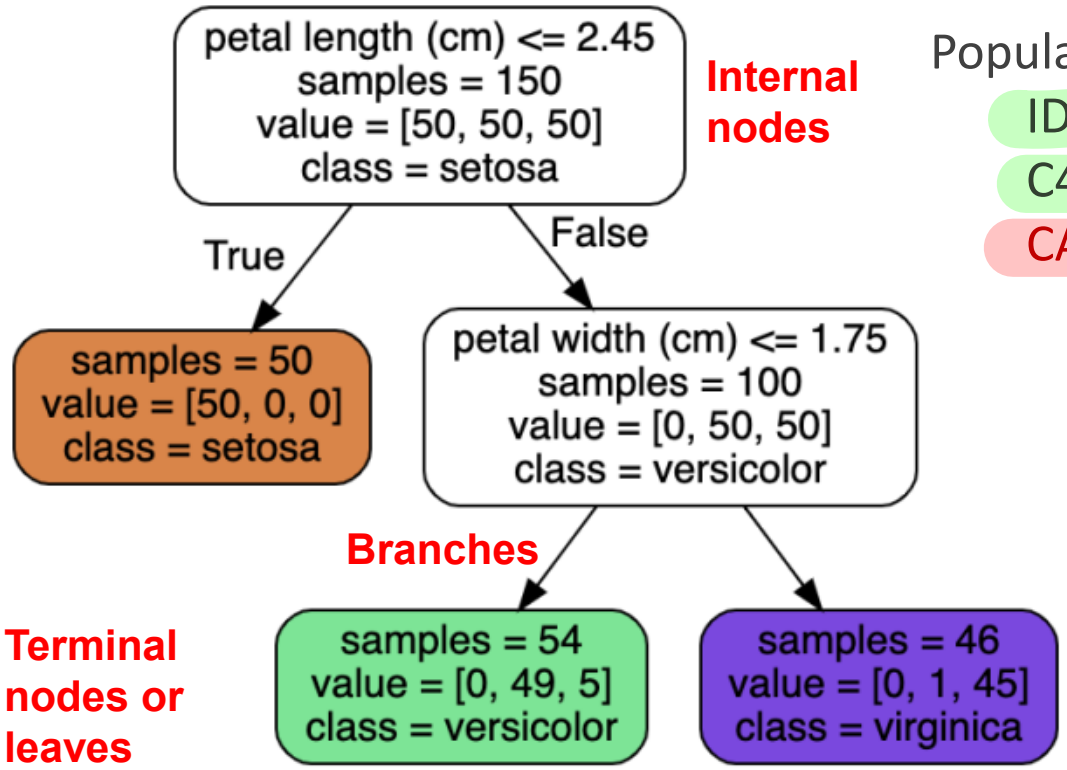
- Interpretability (understanding how models make decisions)



Decision Tree

- A non-parametric supervised learning algorithm for classification and regression

Decision tree



Hunt's Algorithm: Foundation for many decision tree algorithms.

Popular Algorithms:

ID3: Developed by Ross Quinlan; uses entropy and information gain.

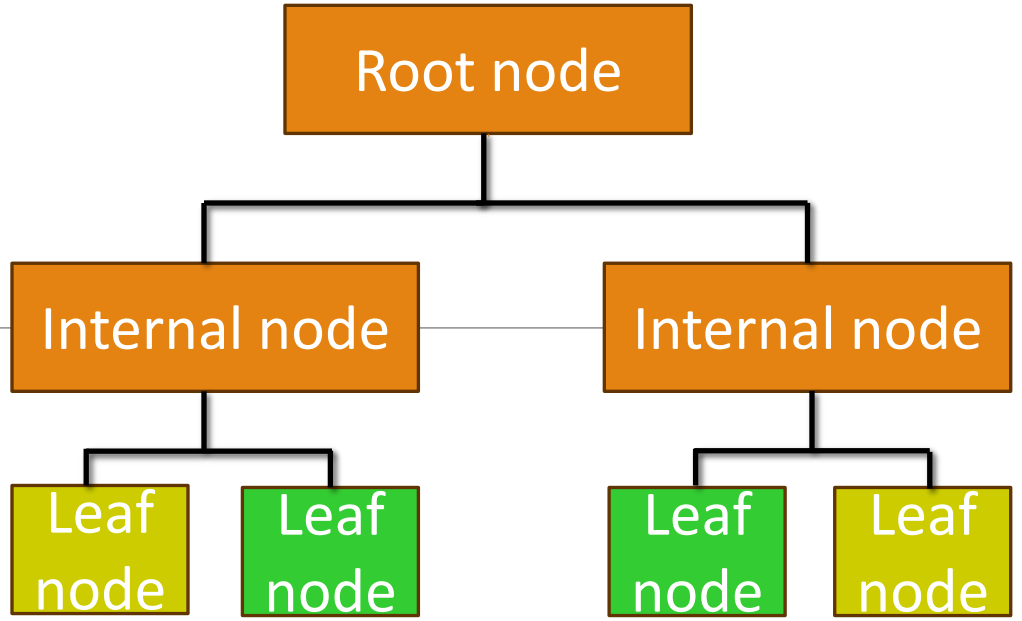
C4.5: An iteration of ID3; uses information gain and gain ratios.

CART: Classification and regression trees; uses Gini impurity for splits.

Nonparametric model

- do not assume a fixed number of parameters
- effective number of parameters of the model can grow with data

Tree based methods



- **Classification and regression trees (CART)**

no fixed number of parameters

- Also, called as decision trees (DTs)
- Used for both classification and regression tasks.
- Nonparametric model

key word

- Decision trees work by recursively dividing the input space into smaller regions and establishing a local model within each of these resulting regions.

key word

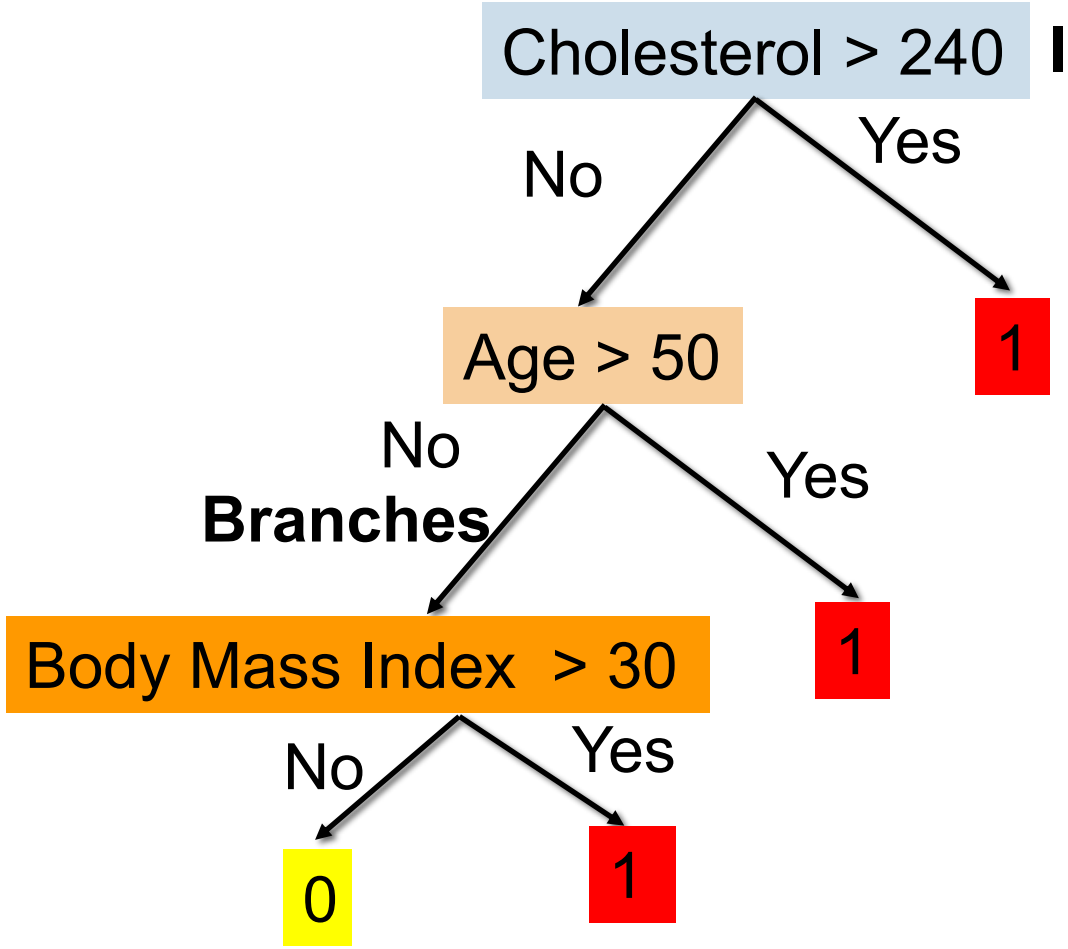
  - **Terminal nodes or leaves** : These are the endpoints of the tree, where a final decision or prediction is made.
    - Classification: leaf represents a class label
    - Regression: leaf provides a predicted numerical value

leaf nodes for each case

  - **Internal nodes**: decision points within the tree. At each internal node, a feature or attribute is examined, and based on a specific condition, the data is divided into subsets.
  - **Branches**: Branches are the connections between nodes in the tree.

Li, Bin, et al. "Classification and regression trees (CART)." *Biometrics* 40.3 (1984): 358-361.

Tree based methods



Internal nodes

Terminal nodes or leaves

Branches

Y Label:
Disease high risk = 1
Disease low risk = 0

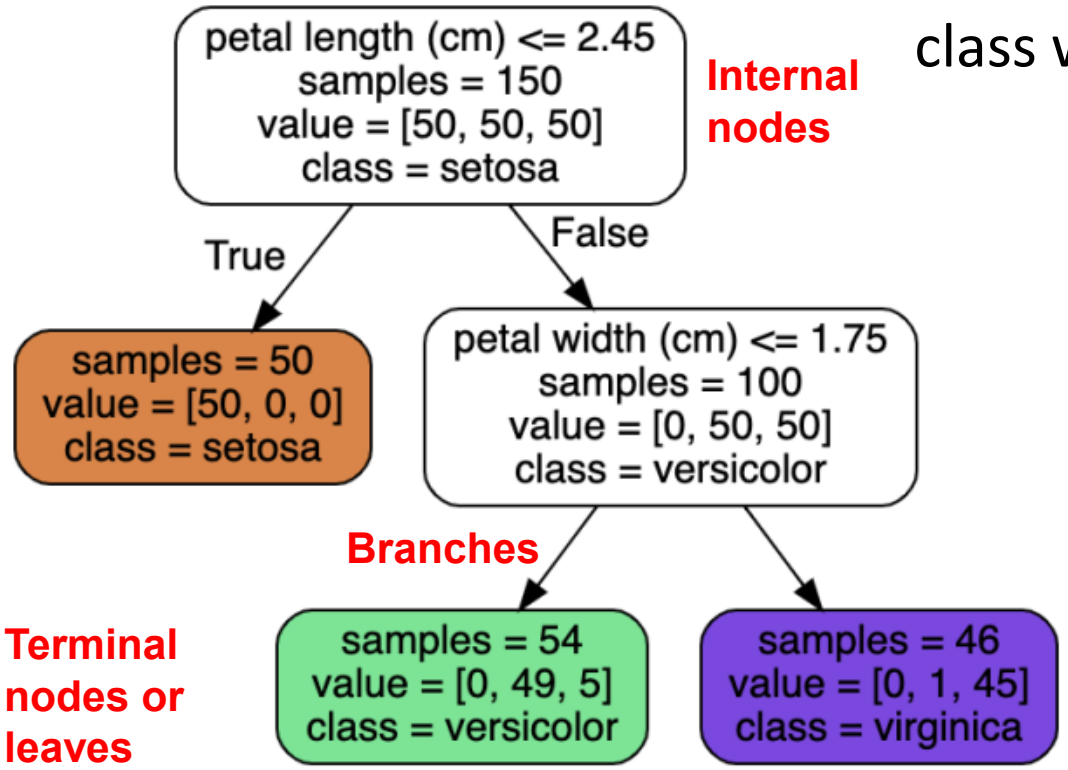
Cholesterol=200
Age=45
BMI =31
Class label ?

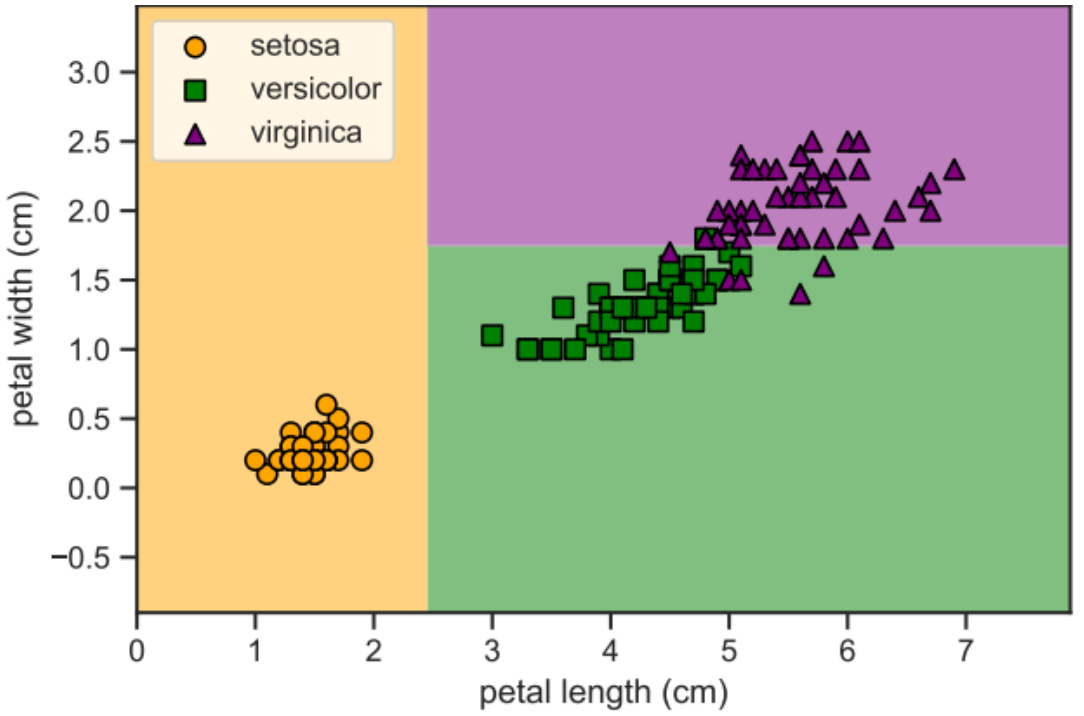
Tree based methods

trees have depth (levels)

- A decision tree with a depth of 2 trained on the iris dataset (only petal length and petal width are used).
- The leaf nodes are color-coded to represent the majority class within each of these nodes.

Decision tree





Decision surface based on the DT

input space portioned into **cuboid regions**

Images from Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

Tree based methods

Advantages

1. Simple to understand
2. Visualizing is straightforward.
3. Little data preparation needed; no normalization, dummy variables, or missing value handling.
4. Predictive cost scales logarithmically with data size.
5. Handles numerical and categorical data.
6. Suitable for multi-output tasks.
7. Offers clear, white-box explanations.
8. Model validation with statistical tests is possible.
9. Performs well even if data deviates from model assumptions

since decision boundaries are discontinuous & non differentiable unable to extrapolate

Disadvantages

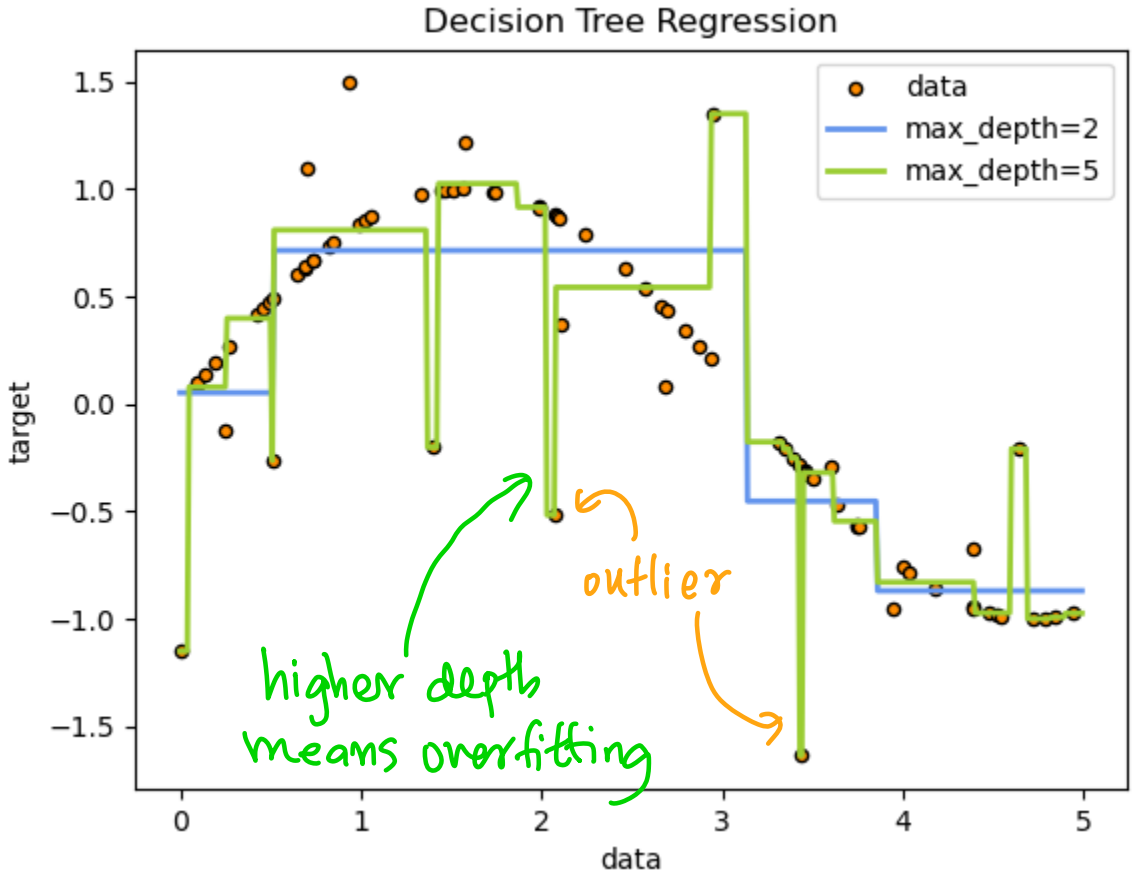
1. DTs tend to become overly complex (overfitting). To combat this, techniques like pruning, specifying minimum leaf samples, or limiting the tree's depth are essential.
2. DTs can exhibit instability due to minor data variations, potentially generating entirely different trees (mitigated by employing decision trees in an ensemble).
3. DTs predictions lack smoothness and continuity; instead, they resemble piecewise constant approximations, making them unsuitable for extrapolation.
4. Learning the optimal decision tree is a computationally challenging problem, often considered NP-complete even for simple concepts.
5. When some classes dominate in the dataset, DT learners tend to produce biased trees (it is advisable to balance the dataset before fitting it with a decision tree).

sensitive to data

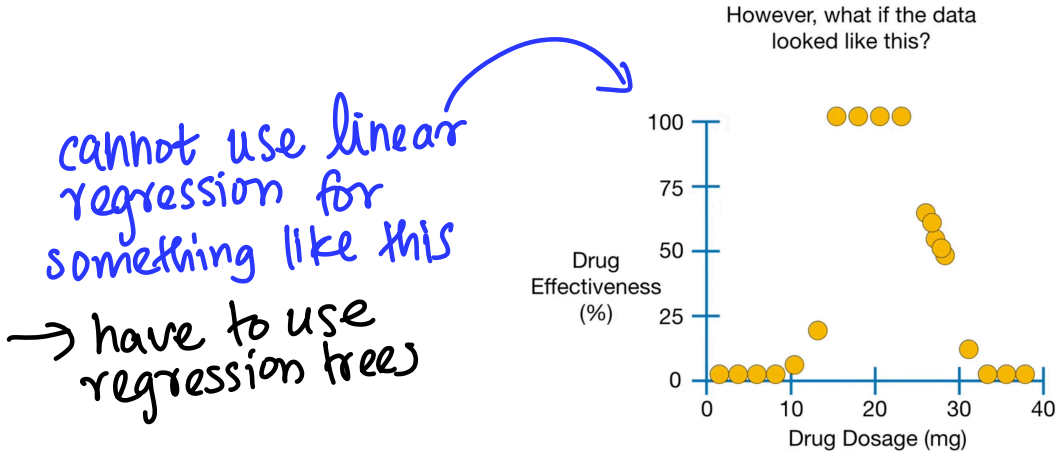
ranges outside of training data

Content adapted from "Decision Trees — scikit-learn 1.3.2 documentation"

Decision Tree Regression



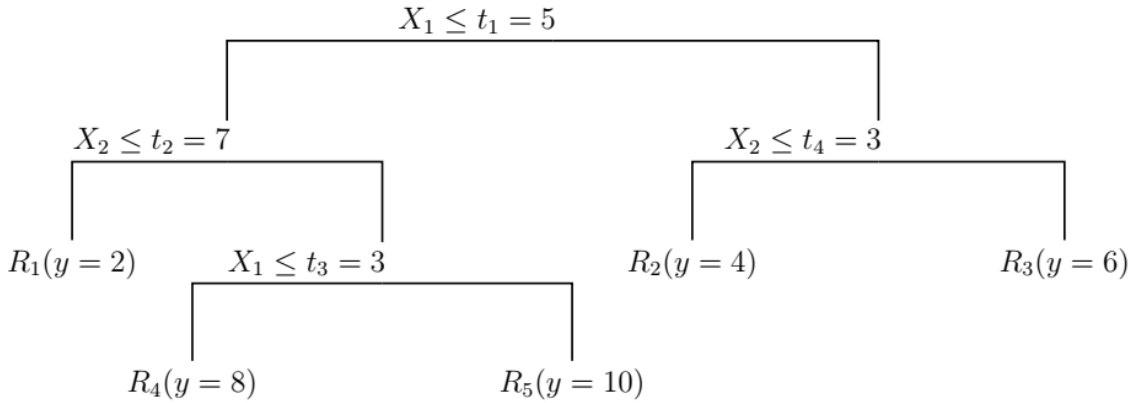
- Decision trees are applied to model a noisy sine curve, resulting in the learning of local linear approximations to the curve.
- Setting a high maximum depth for the tree using the "max_depth" parameter can lead to overfitting, where the tree captures excessive training data details, including noise.



Code Decision Tree Regression — scikit-learn

Model Training

- At every node (i) in the tree, the input feature (d_i) of the vector ($\boldsymbol{x}$) is compared to a threshold (t_i), directing it to the left or right branch based on the comparison.
- The leaves of the tree provide predictions for inputs within their defined region of the input space.



$$R_1 = \{\boldsymbol{x}: x_1 \leq t_1, x_2 \leq t_2\}$$

Partition the predictor space, which includes all potential values for x_1, x_2, ..., x_D, into J separate and **non-overlapping** regions denoted as R_1, R_2, ..., R_J Which minimizes RSS

For each observation in region R_j, prediction is $\hat{y}_{R_j}$.

residual sum of Squares ⟶ $$RSS = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j}\right)^2$$

Need to determine: threshold values

Model definition

Objectives:

- Minimize sum-of-squares error (SSE)/ residual sum of squares (RSS)

$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$$

- Challenges:
  - Feature selection (how many features/ which feature to start)
  - Identify optimal thresholds for selected features
  - Cannot apply SD/SGD ← no parameters to update

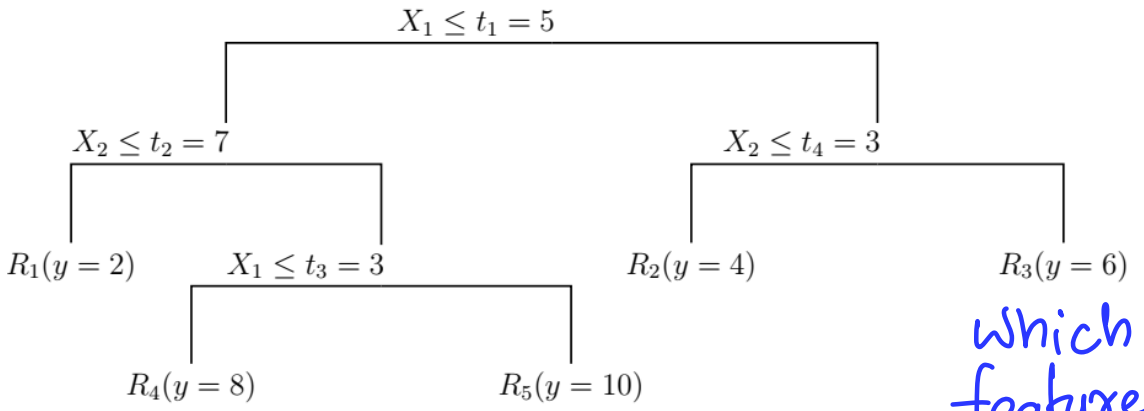
globally optimal solution is impossible

This is computationally infeasible due to the combinatorially large number of possible solutions.

Greedy Optimization is used ← locally optimal solution is achieved

- Start with a single root node (entire input space)
- Grow the tree by adding nodes one at a time and determine the optimal threshold for the chosen variable
- Each candidate region corresponds to the addition of a pair of leaf nodes
- When to stop ?
  - residual error falls below a threshold
  - based on the number of data points in leaf nodes and then prune back the resulting tree

Model Training



$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j}\right)^2$$

Which feature?

Recursive binary splitting

$$R_1(j, s) = \{\boldsymbol{x} | x_j < s\}, \text{and } R_2(j, s) = \{\boldsymbol{x} | x_j \geq s\},$$

What threshold?

Get the value of j and s that minimize

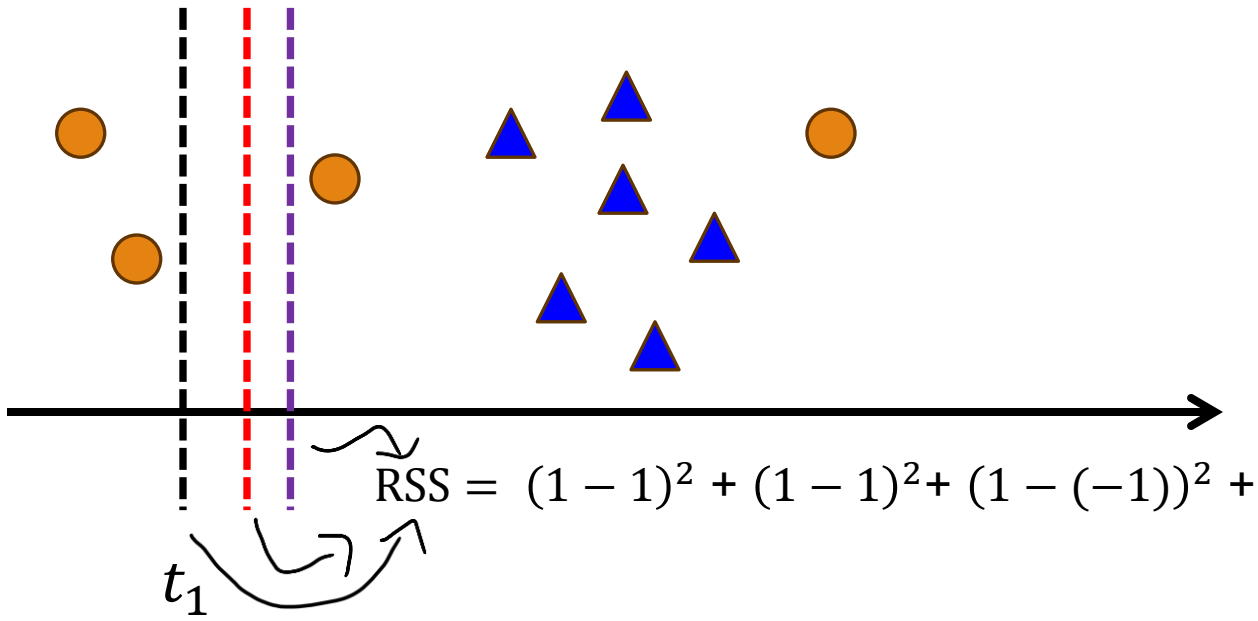
$$\sum_{i: x_i \in R_1(j,s)} \left(y - \hat{y}_{R_1}\right)^2 + \sum_{i: x_i \in R_2(j,s)} \left(y - \hat{y}_{R_2}\right)^2$$

Nonparametric model

- do not assume a fixed number of parameters
- effective number of parameters of the model can grow with data

Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

Model Training



$$\text{RSS} = \sum_{j=1}^{J} \sum_{i \in R_j} \left(y_i - \hat{y}_{R_j} \right)^2$$

y=1

y=-1

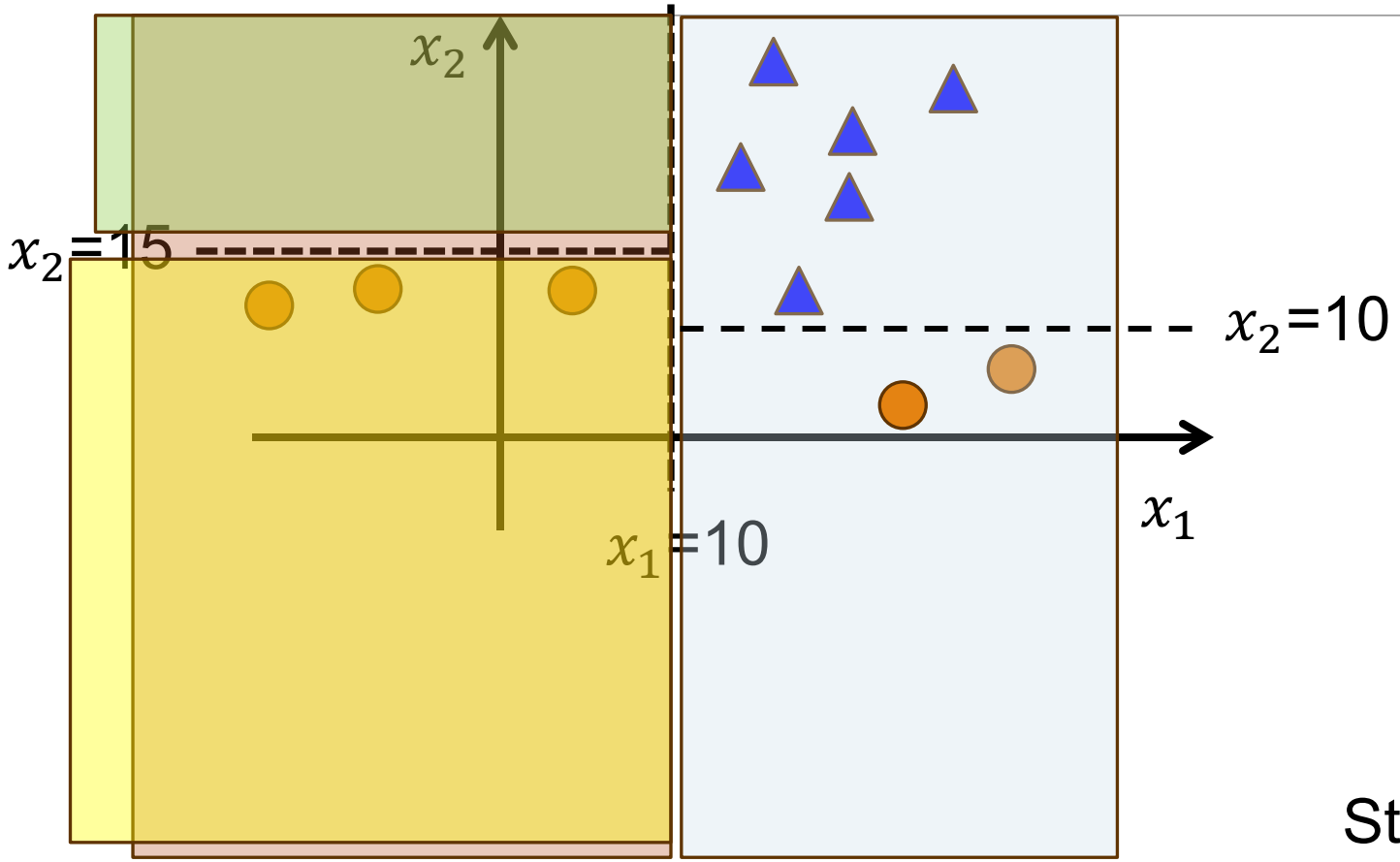
$$\text{RSS} = (1-1)^2 + (1-1)^2 + (1-(-1))^2 + 6 \text{ x } (-1-(-1))^2 + (1-(-1))^2$$

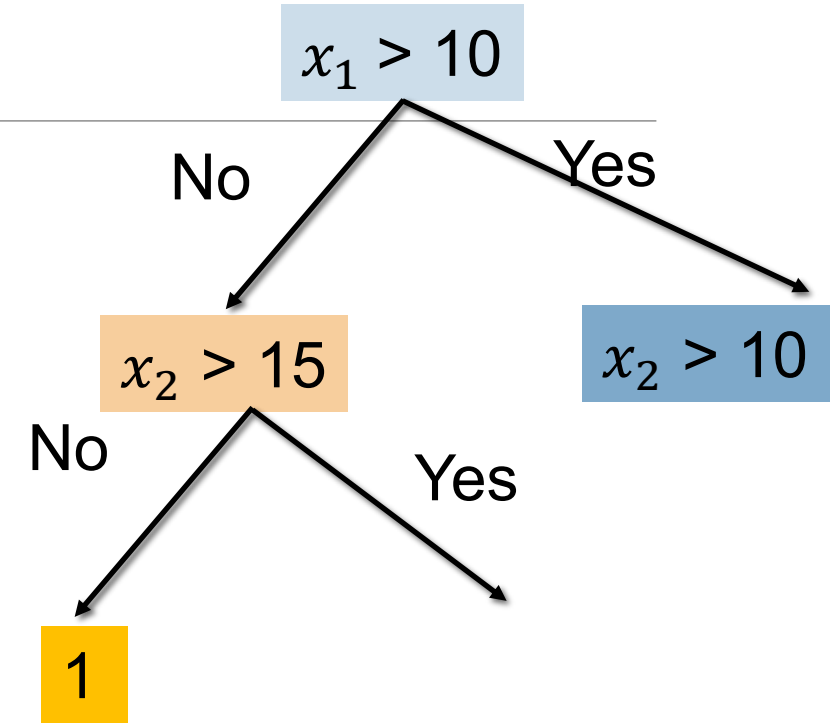
Roughly one split value for a data point

May be average value between two data points

Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

Building a tree





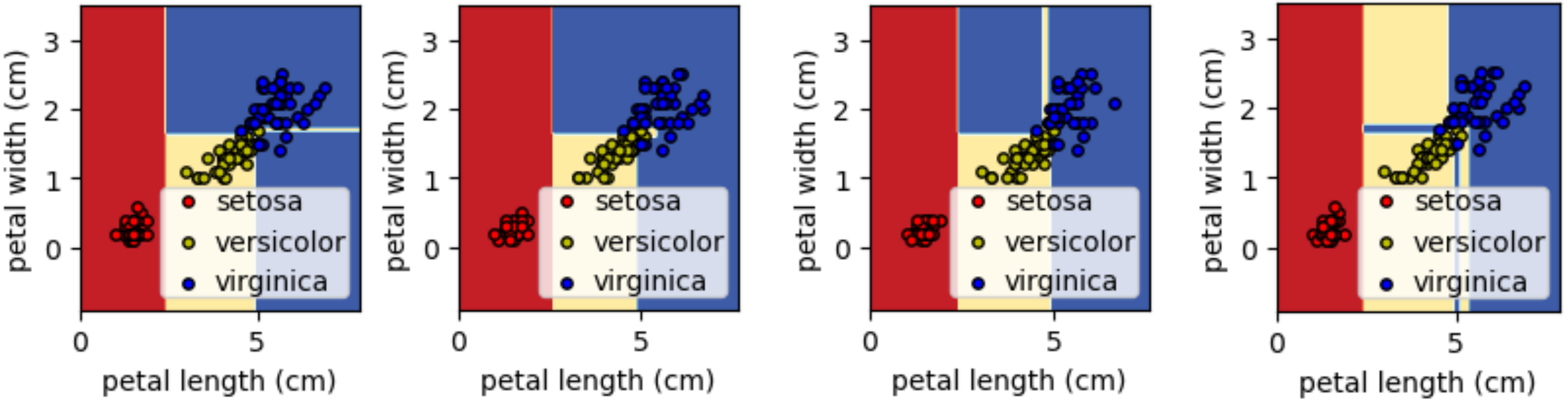
y=1

y=-1

Stopping criteria: number of data points in leaf nodes = 3

Decision Tree

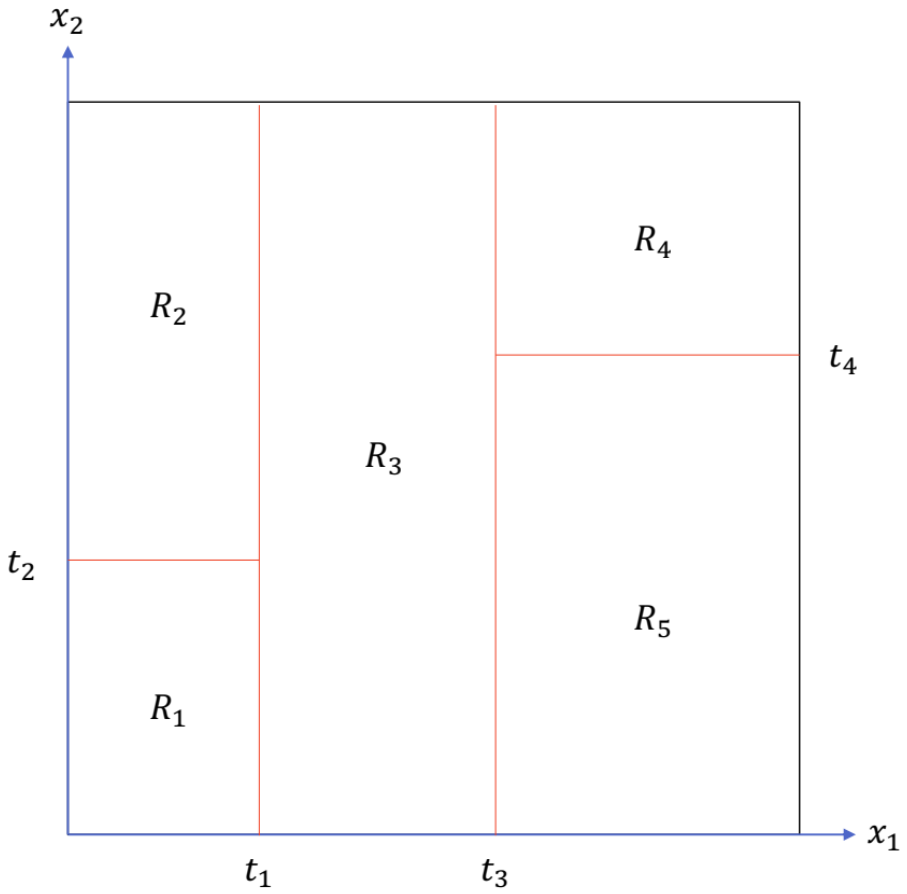
Decision tree- Iris dataset

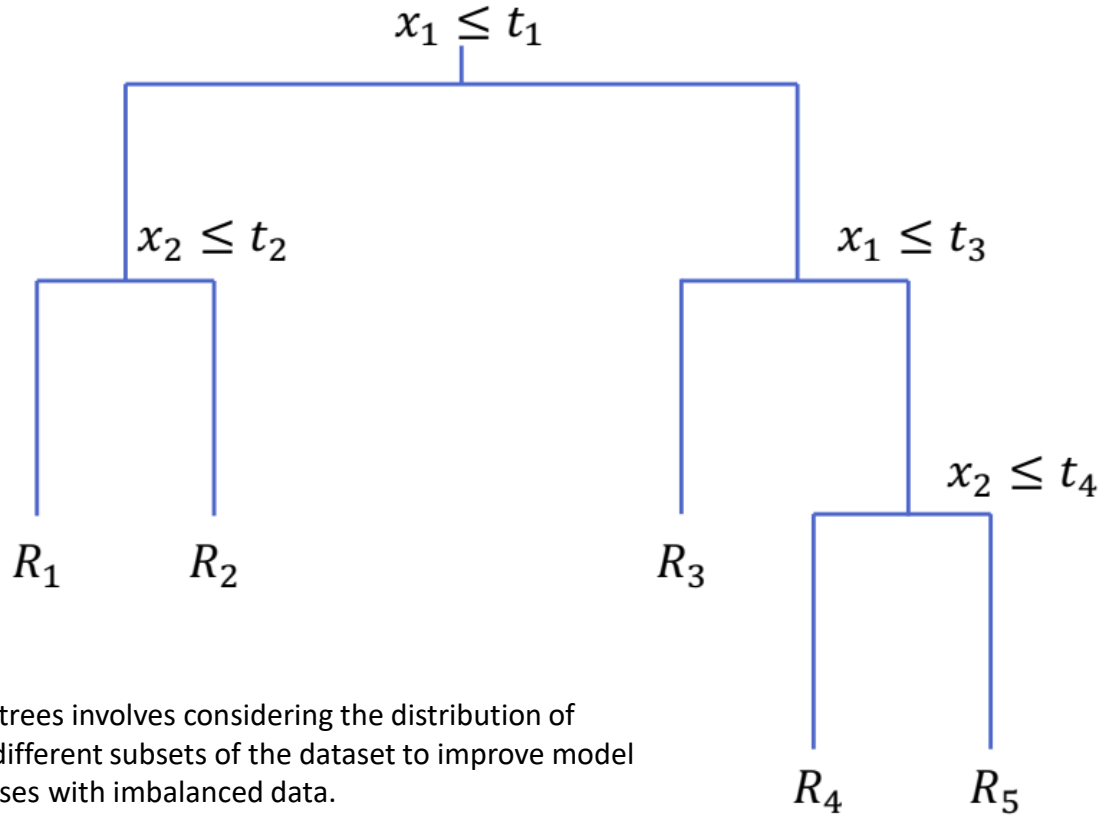


- DT leading to varied predictions when the training data is changed

Stratification

- First split on $x_1 = t_1$.
- If $x_1 < t_1$, split on $x_2 = t_2$.
- If $x_1 > t_1$, split on $x_1 = t_3$.
- If $x_1 > t_3$, split on $x_2 = t_4$.





Stratification in decision trees involves considering the distribution of classes or values within different subsets of the dataset to improve model accuracy, especially in cases with imbalanced data.

regression → averaging

classification → majority voting (committee method)

Ensemble methods

- Decision trees can exhibit instability, leading to varied predictions when the training data is altered/permuted. ➔ high variance estimator.
- To mitigate this variance, ensemble learning, which involves averaging multiple models, is an effective approach.

$$f(y|\boldsymbol{x}) = \frac{1}{|M|} \sum_{m \in M} f_m(y|\boldsymbol{x})$$

(cardinality) no. of models → $|M|$; $m \in M$: for a given model in model collection

- Ensemble learning helps in reducing the instability and variance commonly seen in decision trees.
- In classifier models, it can be more effective to aggregate the outputs by considering a majority vote than averaging. This is known as committee method.
- E.g., binary classification

Number of votes for class 1: $S_1 = \sum_{j=1}^{J} y_j$

$y_j \in \{1,0\}$ be the prediction for the j-th model

Predicted class is class 1 if $S_1 \geq J/2$

Ensemble: combines the predictions of multiple models to produce a single, aggregated prediction.

Ensemble methods

- **Stacking**
- As an alternative to unweighted average or majority vote, is to learn effective methods for combining the base models.
- Combine the base models as (meta-learner or a combiner)

$$f(y|\boldsymbol{x}) = \frac{1}{|M|} \sum_{m \in M} w_m f_m(y|\boldsymbol{x})$$

Weighted average of model outputs
↳ these weights can be learned (by meta learner)

The meta-learner learns how to best use the outputs of the base decision trees to make the final prediction, effectively "stacking" the results of the individual trees.

- It's important to note that in stacking, the combination weights must be trained on a distinct dataset. Without this, the weights could heavily favor the best-performing base model, potentially leading to biased results.

meta-learner should be trained on a seperate dataset (on which no base model has been trained)

Ensemble methods-Bagging

Bagging (**B**ootstrap **Agg**rega**ting**)

- **Bootstrap Sampling**: Bagging begins by creating multiple random subsets (samples) of the training data.
- Each subset is generated by randomly selecting data points with replacement. As a result, some data points may appear in multiple subsets, while others may not appear at all.
- For each bootstrap sample, a base model is trained independently.
- **Aggregation**: Once all base models are trained, bagging aggregates their predictions to make a final prediction.
- The primary benefit of bootstrap is its prevention of the ensemble from over-reliance on any single training example, thus improving overall robustness and generalization.

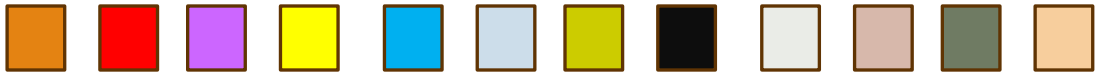
Averaging: reduces variance

Bootstrapping: Numerous training datasets

Ensemble methods-Bagging

Bagging (**B**ootstrap **Agg**rega**ting**)

- **Bootstrap Sampling**: Bagging begins by creating multiple random subsets (samples) of the training data.
- Each subset is generated by randomly selecting data points with replacement. As a result, some data points may appear in multiple subsets, while others may not appear at all.





subset



subset

Tree Pruning

- Unnecessary branches, nodes, or leaves are removed from the tree to simplify its structure and improve its generalization capabilities.
- Improve DTs generalization capabilities.
- Reduce overfitting.
- How to select unnecessary branches, nodes, or leaves ?
  - we may us cross validation to see which tree has the lowest error rate
  - Problem: Lot of tree combinations
- Cost complexity pruning

use metrics → RSS, Gini impurity

similar to L_1 regularization in regression

cost-complexity measure of tree T:

$$R_\alpha = R(T) + \alpha\,|\tilde{T}|$$

loss on validation data

No of leaves

parameter

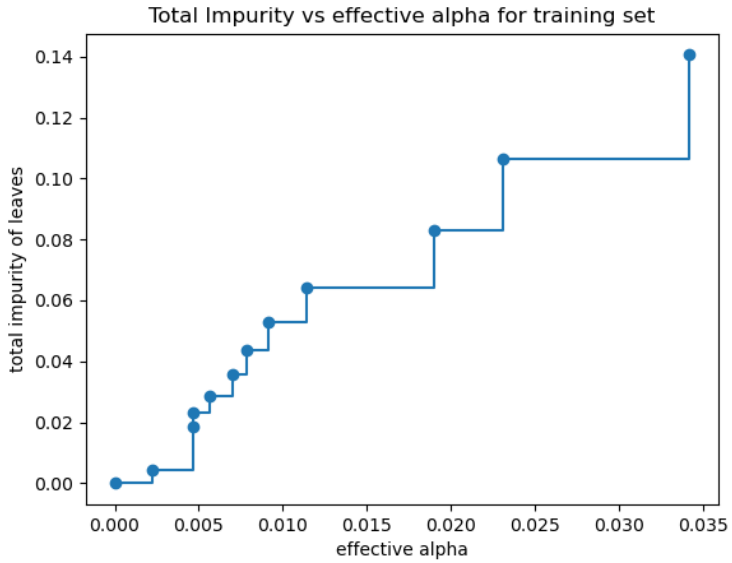
Regularization

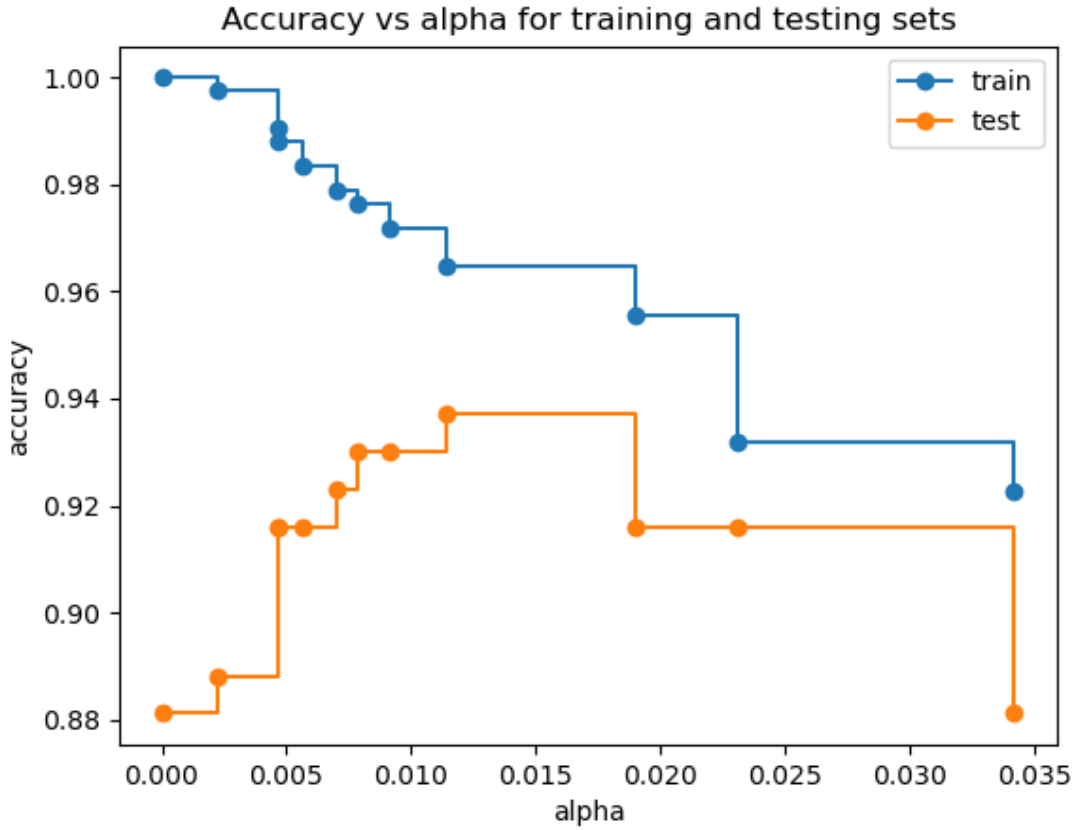
$\alpha = 0$: no pruning, α large: small subtree.

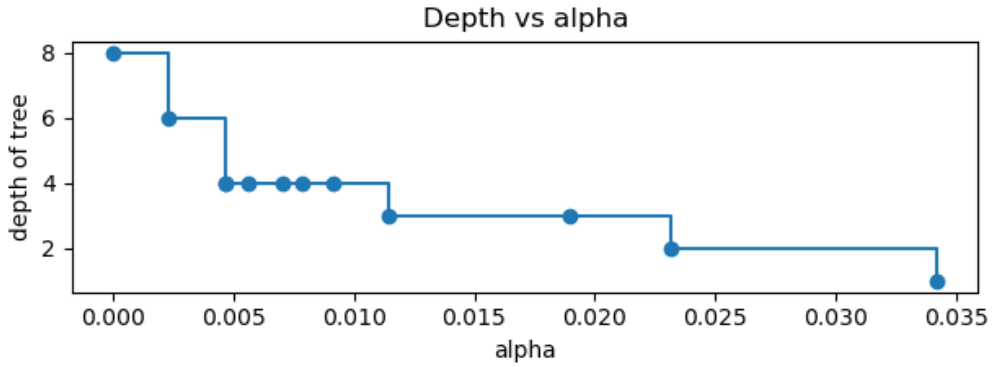
$R(T)$- misclassification error or MSE

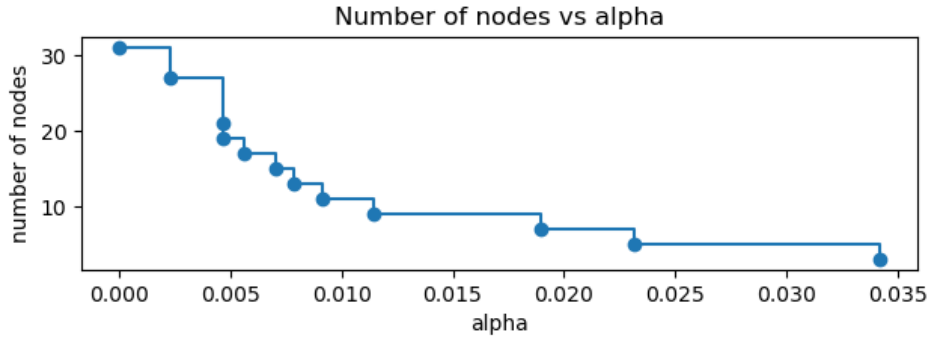
$\tilde{T}$ $number\ of$ terminal nodes

Tree Pruning







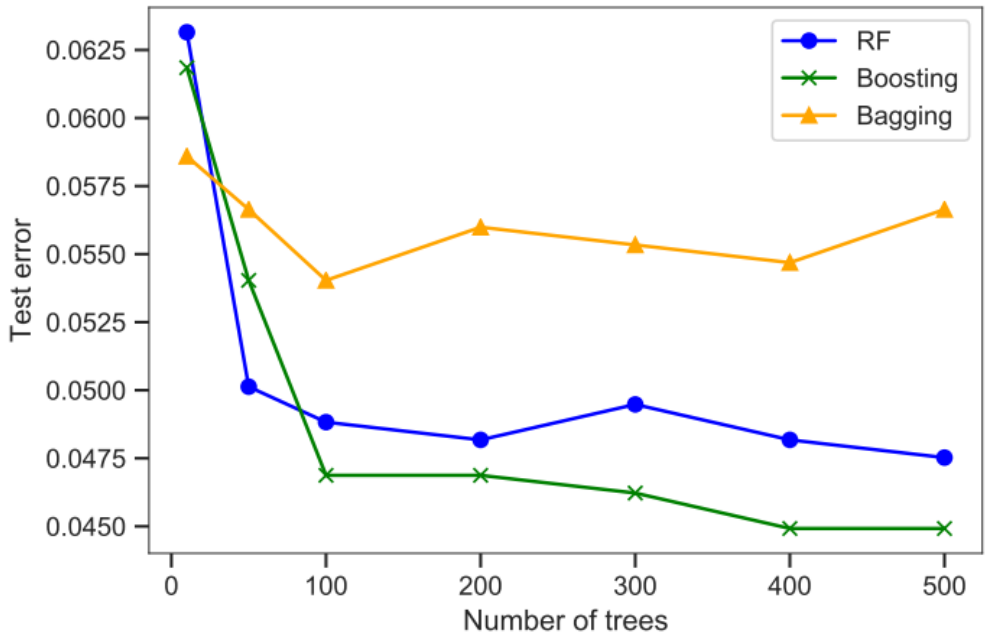


Code: Post pruning decision trees with cost complexity pruning — scikit-learn

Random forests

- Bagging assumes that running the same learning algorithm on various data subsets will produce suitably diverse base models.
- Random forests, a technique that aims to reduce the correlation between base learners, achieves this by building trees using **randomly selected input variables** at each node and randomly **chosen subsets of data sets**.

* drop some input features randomly



random forests work much better than bagged decision trees, because many input features are irrelevant.

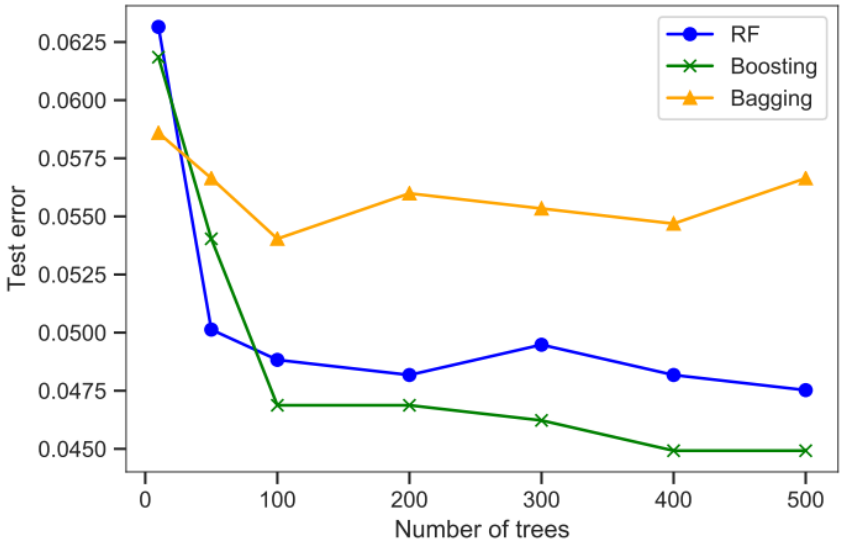
Code: spam_tree_ensemble_compare.ipynb

Kevin P. Murphy, Machine Learning: A Probabilistic Perspective, 2012.

https://www.youtube.com/shorts/hnMz2i_gSCs ↶ Decision trees outperform neural nets on tabular data.

Ensemble methods-Boosting

- Boosting is a sequential ensemble method that focuses on improving the performance of a single base model (usually a weak learner) through iterative training.
- In boosting, each subsequent model is trained to correct the errors of the previous one, and their predictions are combined to create a strong ensemble model.
- Boosting differs from bootstrap sampling as it does not involve creating random subsets of data. Instead, each tree is trained on an adjusted version of the original dataset.
- Common boosting algorithms include AdaBoost, Gradient Boosting (e.g., XGBoost, LightGBM)



* No random subsampling of the dataset

Code: spam_tree_ensemble_compare.ipynb

DTs Examples

- Understanding the decision tree structure
- Multi-output Decision Tree Regression
- Decision trees trained on the iris dataset
- Post pruning decision trees with cost complexity pruning





Thank You

Q & A